Music School Workload Allocation & Resource Scheduling System

Milestone 2: Detailed Design, Test Plan and Wireframe Report

Team Name: P5-5
Lab Group: P5 - Team 5
Submission Date: 20 November 2026

Student Name	Student ID	Academic Role	Contribution %	Primary Authored Sections & Responsibilities
Joseph Poon	2501079	Lead Architect	20%	Final Class Diagram (§1.3), Component Diagram (§2.0), 3-Tier Layered Architecture (§3.1), White-Box Basis Path Testing (§4.2).
Ryan Ke	2501911	Requirements & QA Lead	20%	Final Requirements Matrix & M1 Diff (§1.2), Black-Box Decision Table Testing (§4.1), Non-Plagiarism & AI Declarations.
Wei Xiang	2500686	Detailed Design Lead	20%	Design Pattern Evaluation & GoF Strategy Implementation (§3.2), Architecture Rationale (§3.1), Test Case Derivation.
Tan Yan Qi	2500030	Test & Verification Lead	20%	Interface Specifications (§1.3, §2.0), Black-Box Combinatorial Reduction (§4.1), White-Box CFG Complexity Mathematics (§4.2).
Zayne Lee	2503844	UI/UX & Wireframe Lead	20%	Wireframe Architecture & Interaction Models (§5.0), Traceability Verification Matrix, Presentation Deck Synthesis.

Table of Contents

- Non-Plagiarism Declaration
- AI Usage Declaration
- 1. Introduction
  - 1.1 Team Member Contributions
  - 1.2 Final Requirements Specification
  - 1.3 Final Class Diagram
- 2. Component Diagram
  - 2.1 Subsystem Decomposition & Component Boundaries
  - 2.2 Interface Contracts (Provided & Required)
  - 2.3 Architectural Design Rationale
- 3. Detailed Design
  - 3.1 Software Architecture Design
  - 3.2 Design Pattern Evaluation & Implementation
    - 3.2.1 Pattern Evaluation: Strategy vs. Observer
    - 3.2.2 Selected Implementation: Strategy Pattern
    - 3.2.3 Structural Transformation: Before & After
- 4. Testing Plan & Verification
  - 4.1 Black Box Testing: Decision Table Derivation
  - 4.2 White Box Testing: Basis Path & Cyclomatic Complexity
- 5. Wireframe & User Interaction Design
  - 5.1 Manager Scheduling & Outlier Intelligence View
  - 5.2 Interactive Allocation Matrix & Conflict Feedback View
  - 5.3 Teacher Availability & Timetable Inspection View
  - 5.4 Audit Log Console & Governance View
- 6. References

Non-Plagiarism Declaration

Students are not allowed to copy assessment solutions from another person or other places. It is the students' responsibility to guarantee that their solutions are their own work. Meanwhile, the students must also ensure that their work is not accessible by others. Where such plagiarism is detected, both of the assessments involved will receive ZERO mark.

We hereby declare that:

- We fully understand and agree to the abovementioned policy.
- We did not copy any materials from others or from other places.
- We did not share our materials with others or upload to any other places for public access.
- We agree that we will not disclose any information or material of the team project to others or upload to any other places for public access.
- We agree that our project will receive Zero mark if any misalignment with the abovementioned policies is detected.

Declared by: Group P5-5
Team Members:

1. Joseph Poon (2501079)
2. Ryan Ke (2501911)
3. Wei Xiang (2500686)
4. Tan Yan Qi (2500030)
5. Zayne Lee (2503844)

Date: 20 November 2026

AI Usage Declaration

While it is not prohibited to use AI for the team project, all teams must use AI tools critically and ensure the learning outcomes are achieved.

In accordance with academic integrity guidelines, generative AI tools (Claude 3.7 Sonnet and Google Gemini 2.5 Pro) were utilized strictly as computational assistants to:

1. **Formatting & Documentation Compilation:** Transform raw technical notes, tabular markdown specifications, and mathematical formulas into Google Docs-compliant typographical markdown and MathJax equations.
2. **UML Syntax Assistance:** Validate PlantUML grammar and layout positioning hints for complex multi-component diagrams.
3. **Boolean Reduction & Cyclomatic Verification:** Cross-check algebraic truth-table prime implicant reductions and verify graph edge-node counts ($E - N + 2P$) for white-box testing.

Critical Human Engineering Interventions: All domain constraints, architectural decisions, interface contracts, decision-table causes/effects, basis path derivations, and software requirement statements were conceived, formulated, and verified directly by the human engineering team. AI-generated layout drafts were reviewed, corrected, and reconciled against Singapore Ministry of Manpower (MOM) regulatory requirements and specific stakeholder interview data.

1. Introduction

1.1 Team Member Contributions

Milestone 2 represents the transition from requirements engineering and domain modeling into detailed architectural decomposition, formal component design, rigorous test engineering, and user interaction prototyping. The engineering responsibilities were distributed across five functional leads:

- **Joseph Poon (Lead Architect):** Authored the UML 2.5 Component Diagram (§2.0), formulated the Service Interface Contracts, updated the Final Class Diagram (§1.3) incorporating interface realizations and running-hour attributes, and designed the White-Box Testing Control Flow Graph (§4.2).
- **Ryan Ke (Requirements & QA Lead):** Spearheaded the M1 feedback reconciliation, restructured the master Functional Requirements into 21 atomic statements satisfying the 7 traits of good requirements (§1.2), developed the Black-Box Decision Table methodology, and performed test data reduction (§4.1).
- **Wei Xiang (Detailed Design Lead):** Conducted architectural pattern analysis (§3.1), evaluated and justified design pattern alternatives (§3.2.1), formalized the GoF Strategy Pattern implementation for candidate teacher tie-breaking (§3.2.2), and authored the before/after structural excerpts (§3.2.3).
- **Tan Yan Qi (Test & Verification Lead):** Formulated interface method contracts for inter-component communication (§2.2), mathematically validated basis path coverage and cyclomatic complexity formulas (§4.2), and synthesized black-box test suites covering nominal, boundary, and error cases.
- **Zayne Lee (UI/UX & Wireframe Lead):** Designed responsive wireframe user journeys (§5.0) matching all finalized use cases, established requirement-to-prototype traceability, and coordinated presentation artifact preparation.

1.2 Final Requirements Specification

Following consultation feedback from Milestone 1, the requirements inventory was thoroughly audited against the **7 Traits of a Good Requirement** (Unambiguous, Atomic, Complete, Consistent, Verifiable, Modifiable, and Traceable).

Key Improvements & Refinements from Milestone 1:

1. **Elimination of Scope Creep (Peer Lesson Swap):** Former requirement `FR-11` (Bilateral Peer Lesson Swap) was officially decommissioned and relegated to Table 2.2 (*Won't-have Requirements*). Client operating policy mandates centralized managerial governance over timetable adjustments to ensure equitable workload balance and prevent uncoordinated schedule churn.
2. **Decomposition of Compound Statements:** Multi-part requirements (such as outlier identification and multi-criteria tie-breaking) were separated into atomic, independently testable specifications.
3. **Clarification of Ambiguous Terminology:** Subjective phrases were replaced with concrete operational metrics:
    - *"Workload Deficit"* is formally defined as the non-negative difference between a teacher's statutory 40.0-hour weekly target and their current running weekly hours ($\text{Deficit} = \max(0, 40.0 - \text{runningWeeklyHours})$).
    - *"Fatigue Clustering"* is defined as back-to-back lesson assignments up to a strict ceiling of 4.0 continuous hours, followed by an obligatory minimum 1.0-hour rest interval.
    - *"Availability Pre-Condition"* requires teachers to submit at least 60.0 hours of available time per week, establishing an adequate scheduling buffer for managerial allocation.
4. **Purging of UI Styling from Functional Requirements:** Visual color specifications (amber/red indicators) were removed from `FR-02` and consolidated into `NFR-USE-02`.
5. **Formalization of Audit Log Governance:** Elevated audit trail recording and inspection into dedicated, traceable functional specifications (`FR-21`) mapped directly to `UC-11`, `AuditLogUI`, and `AuditLogController`.

Table 1.1: Final Master Functional Requirements Specification (FR-01 to FR-21)

ID	P	Functional Requirement Specification (System Behavior & Acceptance Criteria)	Mapped UC	Update from M1
FR-01	**M**	**Workload Running Hours:** The system shall compute and display running weekly and monthly allocated teaching hours for each active teacher, updating immediately upon schedule confirmation.	UC-01	Refined attribute mapping to `runningWeeklyHours` and `runningMonthlyHours`.
FR-02	**M**	**Lowest-Workload Identification:** The system shall identify and display the three active teachers with the lowest total allocated weekly teaching hours.	UC-01	Atomized: isolated sorting behavior from visual color alerts.
FR-03	**M**	**Over-Allocation Threshold Warning:** The system shall flag each teacher whose allocated weekly hours meet or exceed the statutory threshold of 40.0 hours.	UC-01	Atomized: separated threshold alerting from candidate ranking.
FR-04	**M**	**Lesson Slot Allocation:** The system shall allow managers to assign an unassigned 30-minute lesson slot to an eligible teacher and available studio for a target week.	UC-02	Reframed from batch weekly allocation to granular per-slot allocation.
FR-05	**M**	**Candidate Teacher Comparative Display:** The system shall display up to three qualified candidate teachers side-by-side during slot allocation, presenting running weekly workload, availability status, and physical studio location.	UC-02	Clarified candidate presentation parameters; removed vague "dynamic surface" phrasing.
FR-06	**S**	**Allocation Tie-Breaking & Ranking:** The system shall rank candidate teachers in descending order of workload deficit ($\max(0, 40.0 - \text{runningWeeklyHours})$), breaking ties using matching declared day and timeslot preferences.	UC-02	Replaced vague "preferred conditions" with explicit mathematical deficit and day/time preferences.
FR-07	**M**	**1-Month Availability Submission:** The system shall allow teachers to submit recurring 30-minute availability slots up to 1 month ahead, enforcing a minimum declaration of 60.0 available hours per week to ensure scheduling flexibility.	UC-07	Introduced mandatory 60.0h minimum availability rule to resolve preference-availability redundancy.
FR-08	**M**	**Pre-Cutoff Availability Modification:** The system shall allow teachers to modify submitted availability slots without managerial approval until 23:59 hours on the 19th calendar day of the preceding month.	UC-07	Clarified strict temporal boundary for autonomous staff edits.
FR-09	**M**	**Weekly Schedule Preferences:** The system shall allow teachers to specify preferred teaching days and preferred lesson clustering up to a maximum of 4 consecutive hours, followed by a mandatory minimum 1-hour rest interval.	UC-08	Excised subjective "acoustic preference"; formalized 4h teaching block ceiling and 1h rest buffer.
FR-10	**M**	**Personal Schedule Inspection:** The system shall provide teachers with an individual schedule view displaying confirmed weekly lesson assignments and cumulative monthly teaching hours.	UC-09	Stripped UI viewport and PII masking clauses (relocated to NFRs).
FR-11	**M**	**Lesson Rejection Submission:** The system shall record a teacher's lesson rejection request along with a mandatory justification reason.	UC-10	Atomized: isolated rejection capture from exception queue routing.
FR-12	**M**	**Rejection Exception Queue Routing:** The system shall route confirmed lesson rejections to the manager exception queue for substitute re-allocation.	UC-03	Explicitly formalized queue handoff to reallocation workflow.
FR-13	**S**	**Late Availability Review:** The system shall flag availability modifications submitted after 23:59 on the 19th as pending exceptions and route them to the manager exception queue.	UC-04	Standardized exception adjudication lifecycle.

FR-14	**S**	**Studio Maintenance Blackout:** The system shall allow IT administrators to schedule maintenance blackout windows for studios, automatically suppressing blacked-out rooms from lesson allocation.	UC-15	Retained; verified conflict suppression integration.
FR-15	**S**	**Emergency Absence Substitution:** The system shall allow managers to reallocate all scheduled lessons of an incapacitated teacher to qualified available substitute teachers.	UC-06	Reframed as a multi-slot invocation of the core slot allocation engine.
FR-16	**M**	**User Lifecycle Management:** The system shall allow IT administrators to provision user accounts and execute logical deactivations for Teachers, Managers, and Administrators without deleting historical records.	UC-12, UC-13	Replaced physical deletion with explicit logical status deactivation.
FR-17	**M**	**Studio Configuration & Drum Restriction:** The system shall manage configuration parameters for Studios 1–5, strictly enforcing the rule that drum lessons can only be allocated to Studio 5 (equipped with acoustic isolation).	UC-14	Retained; formalized physical acoustic isolation rule.
FR-18	**M**	**Weekly Roster Publishing:** The system shall allow managers to transition draft weekly rosters to published status and dispatch alert notifications to assigned teachers via the external notification service.	UC-05	Linked to External Notification Service actor.
FR-19	**S**	**Studio Utilization Analytics:** The system shall calculate booked versus available capacity across all 86 weekly operating hours and export utilization reports for management.	UC-16	Retained; clarified 86-hour operating baseline.
FR-20	**M**	**Labor Compliance Monitoring:** The system shall track teacher working hours against the statutory 44.0-hour standard (Employment Act Part IV) and require explicit managerial justification for authorized overtime.	UC-17	Aligned with statutory 44.0h ceiling and advisory tier prompts.
FR-21	**M**	**Immutable Audit Trail & Inspection:** The system shall record an immutable audit log entry for every allocation, managerial override, and administrative status change, and allow authorized users to query the audit trail.	UC-11	New dedicated specification replacing deprecated peer swap; traces to UC-11.

Table 1.2: Quantifiable Non-Functional Requirements (Standardized System Perspective)

NFR ID	Category	Metric Threshold & Quantifiable Operational Condition	Statutory / Operational Justification
NFR-USE-01	Usability	The user interface shall remain fully operable across viewports from 375px (mobile) to 1920px (desktop) without horizontal scrolling or control clipping.	Enables mobile schedule review for teachers and desktop allocation for managers.
NFR-USE-02	Usability	The user interface shall render workload indicators using 3 distinct visual tiers: Green (<35.0h), Amber (35.0h–40.0h), and Red (>40.0h), combining distinct color hues with text badges and iconography in accordance with WCAG 2.2 AA Rule 1.4.1.	Prevents over-allocation and ensures accessibility for color-deficient users (relocated from FR-02).
NFR-USE-03	Usability	The user interface shall size all interactive touch targets on mobile viewports to at least 44x44 CSS pixels.	Complies with mobile ergonomics for active instructors between sessions.
NFR-PER-01	Performance	The system shall achieve a First Contentful Paint of <= 1.0s and full Page Render of <= 1.5s under 20 concurrent administrative users on standard broadband.	Ensures responsive management operations during peak monthly scheduling cycles.
NFR-PER-02	Performance	The system shall execute the multi-constraint clash validation engine and return feedback within 300ms upon slot placement.	Delivers seamless interactive decision feedback without interface stutter.
NFR-PER-03	Performance	The system shall propagate studio configuration modifications to the scheduling validation engine within 500ms of persistence.	Guarantees immediate constraint enforcement without requiring manual browser reloads.
NFR-REL-01	Reliability	The system shall maintain operational availability of >= 99.5% uptime during monthly scheduling windows (19th to 24th of each calendar month).	Prevents administrative bottlenecks during high-stakes roster generation.
NFR-REL-02	Reliability	The system shall perform automated daily cryptographic database snapshots to guarantee a Recovery Point Objective (RPO) <= 24h and Recovery Time Objective (RTO) <= 2h.	Protects academy scheduling history and compliance audit data from catastrophic loss.
NFR-REL-03	Reliability	The system shall execute account and lesson record removals exclusively via logical status flags, prohibiting physical database deletions.	Maintains unbroken historical auditability and data integrity.
NFR-REL-04	Reliability	The system shall cache unsubmitted form drafts in local browser session storage, preserving availability edits during disconnections <= 5 minutes.	Protects teachers against data loss during transient campus Wi-Fi dropouts.
NFR-SEC-01	Security	The system shall encrypt all personal data at rest and in transit, and mask student phone numbers and contact details in all instructor-facing views.	Mandatory compliance with Singapore Personal Data Protection Act (PDPA Section 24).
NFR-SEC-02	Security	The system shall enforce role-based access control to prevent teachers from accessing peer schedules, financial records, or administrative panels.	Protects employee privacy and prevents unauthorized schedule modifications.
NFR-COM-01	Compliance	The system shall preserve immutable audit records for at least 24 calendar months verified with SHA-256 cryptographic hashes.	Satisfies regulatory labor record retention rules (Ministry of Manpower, 2026).
NFR-COM-02	Compliance	The system shall monitor cumulative weekly hours against the statutory 44.0-hour standard (Employment Act Part IV Section 38) and require managerial justification for authorized overtime.	Mitigates institutional legal liability while supporting legitimate overtime coverage.

1.3 Final Class Diagram

The final domain class model has been upgraded from Milestone 1 to establish full alignment with the **UML 2.5 Component Diagram** (§2.0) and the **3-Tier Layered Architecture** (§3.1).

Core Structural Enhancements:

1. **Introduction of Formal Service Interfaces:** Pure abstract interface classes (`<<interface>>`) were introduced to decouple the boundary tier from control implementations:
    - `IScheduleService`: Exposes lesson slot allocation, emergency substitution, and roster publishing.
    - `IConflictValidator`: Decouples multi-constraint validation logic from scheduling orchestration.
    - `IFacultyService`: Governs availability submissions, preference capture, and workload intelligence.
    - `IFacilityService`: Manages studio metadata, room maintenance blackouts, and utilization reporting.
    - `IAuditLogService`: Standardizes immutable audit trail recording and query interfaces.
    - `ITieBreakerStrategy`: Realizes the GoF Strategy Pattern for dynamic candidate teacher ranking.
2. **Explicit Running-Hour State Tracking:** `Teacher` now encapsulates private attributes `runningWeeklyHours` and `runningMonthlyHours`, along with public mutators `updateRunningHours(deltaHours: float)` and accessor methods, directly resolving M1 feedback regarding workload calculation transparency.
3. **Strict Entity-Control-Boundary (ECB) Layering:** Boundaries invoke only service interfaces or controllers; controllers interact with entities; entities encapsulate state and business invariants. Boundary-to-Entity leaks have been completely eliminated.

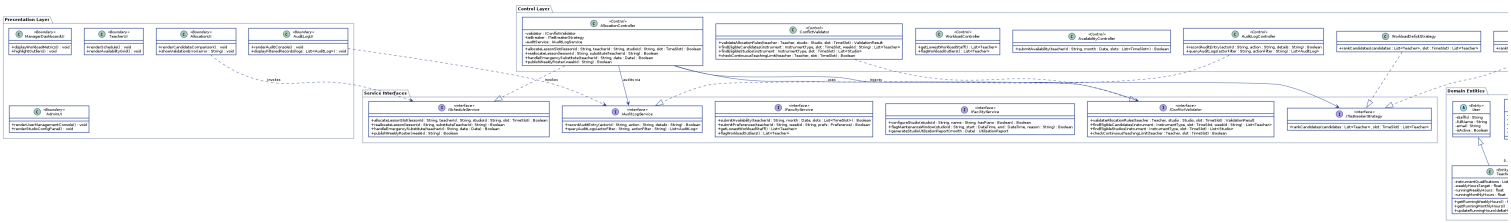


Figure 1.1: Final Domain Class Diagram with Interface Realizations

2. Component Diagram

2.1 Subsystem Decomposition & Component Boundaries

In accordance with UML 2.5 standards, the system is decomposed into highly cohesive, loosely coupled subsystems organized around **business capabilities** rather than technical layers. Each component encapsulates its internal controllers and domain entities, exposing functionality strictly through well-defined **provided interfaces (lollipops)** and consuming external capabilities via **required interfaces (sockets)**.

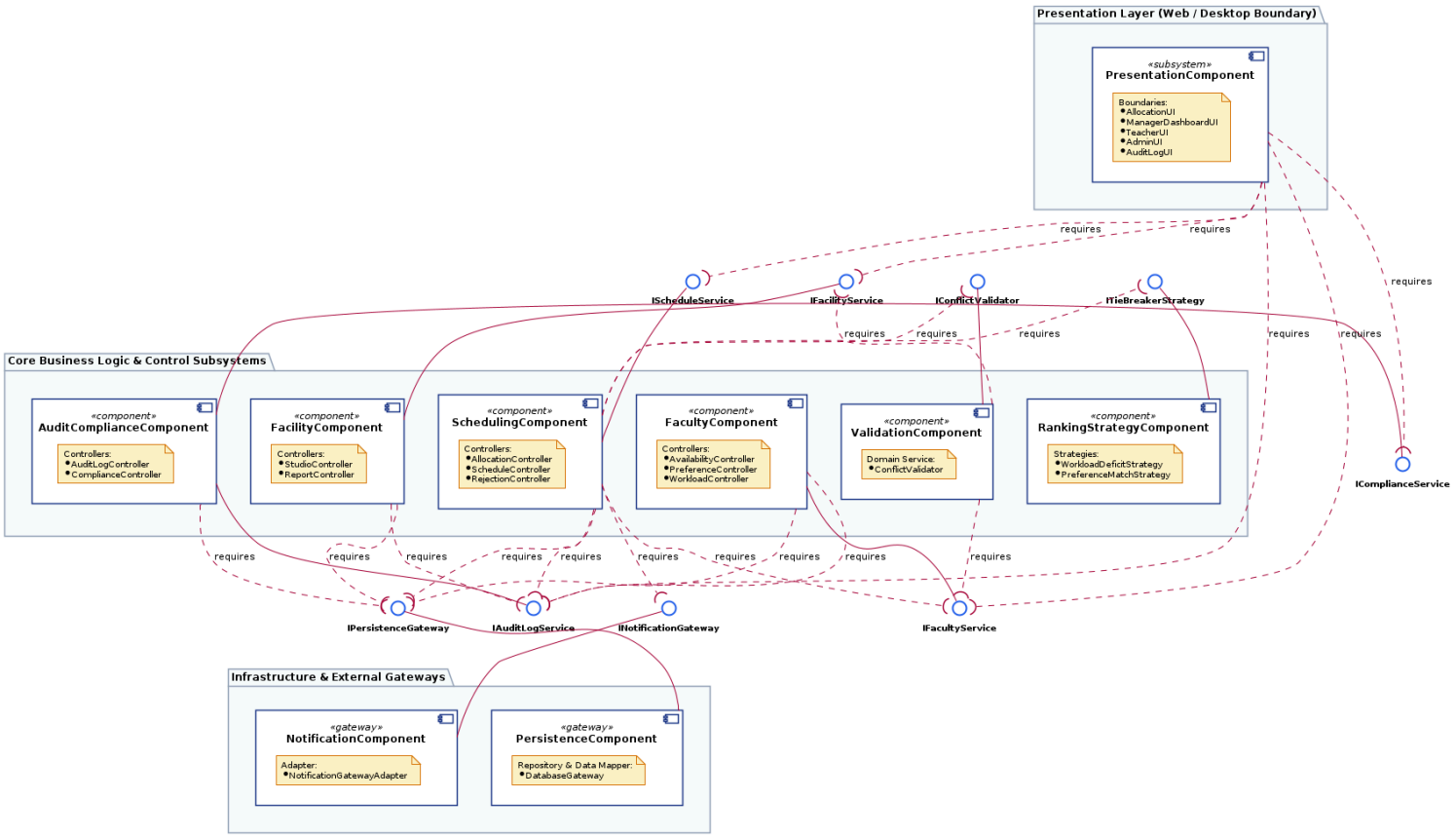


Figure 2.1: UML 2.5 Component Diagram for Workload Allocation System

---

2.2 Interface Contracts (Provided & Required)

The component model defines explicit interaction contracts, enforcing structural modularity and enabling independent testability:

1. `IScheduleService` (Provided by `SchedulingComponent`)

- **Primary Responsibility:** Manages lesson lifecycle transitions, timetable persistence, and draft-to-published roster promotion.
- **Key Operations:**
  - `allocateLessonSlot(lessonId: String, teacherId: String, studioId: String, slot: TimeSlot): Boolean`
  - `reallocateLesson(lessonId: String, substituteTeacherId: String): Boolean`
  - `handleEmergencySubstitute(teacherId: String, date: Date): Boolean`
  - `publishWeeklyRoster(weekId: String): Boolean`
- **Required Interfaces:** Consumes `IConflictValidator` for rule enforcement, `ITieBreakerStrategy` for candidate ordering, `IAuditLogService` for compliance logging, and `INotificationGateway` for alert dispatch.

2. `IConflictValidator` (Provided by `ValidationComponent`)

- **Primary Responsibility:** High-performance domain validation engine executing multi-constraint checks prior to allocation commit.
- **Key Operations:**
  - `validateAllocationRules(teacher: Teacher, studio: Studio, slot: TimeSlot): ValidationResult`

    - `findEligibleCandidates(instrument: InstrumentType, slot: TimeSlot, weekId: String): List<Teacher>`
    - `findEligibleStudios(instrument: InstrumentType, slot: TimeSlot): List<Studio>`
    - `checkContinuousTeachingLimit(teacher: Teacher, slot: TimeSlot): Boolean`
- **Required Interfaces:** Consumes `IFacultyService` and `IFacilityService` to inspect current schedules and blackout states.

3. `IFacultyService` (Provided by `FacultyComponent`)
- **Primary Responsibility:** Governs teacher profile lifecycles, monthly availability grids, scheduling preferences, and real-time workload tallies.
- **Key Operations:**
    - `submitAvailability(teacherId: String, month: Date, slots: List<TimeSlot>): Boolean`
    - `submitPreferences(teacherId: String, weekId: String, prefs: Preference): Boolean`
    - `getLowestWorkloadStaff(): List<Teacher>`
    - `flagWorkloadOutliers(): List<Teacher>`
- **Required Interfaces:** Consumes `IAuditLogService` and `IPersistenceGateway`.

4. `IFacilityService` (Provided by `FacilityComponent`)
- **Primary Responsibility:** Maintains physical room metadata, acoustic equipment flags (Studio 5 drum kit), servicing blackouts, and utilization analytics.
- **Key Operations:**
    - `configureStudio(studioId: String, name: String, hasPiano: Boolean): Boolean`
    - `flagMaintenanceWindow(studioId: String, start: DateTime, end: DateTime, reason: String): Boolean`
    - `generateStudioUtilizationReport(month: Date): UtilizationReport`
- **Required Interfaces:** Consumes `IAuditLogService` and `IPersistenceGateway`.

5. `IAuditLogService` & `IComplianceService` (Provided by `AuditComplianceComponent`)
- **Primary Responsibility:** Appends append-only cryptographic transaction logs and audits statutory labor hour thresholds under Singapore Employment Act Part IV.
- **Key Operations:**
    - `recordAuditEntry(actorId: String, action: String, details: String): Boolean`
    - `queryAuditLogs(actorFilter: String, actionFilter: String): List<AuditLog>`
    - `auditStatutoryLaborCompliance(): ComplianceReport`
- **Required Interfaces:** Consumes `IPersistenceGateway`.

6. `ITieBreakerStrategy` (Provided by `RankingStrategyComponent`)
- **Primary Responsibility:** Encapsulates algorithmic sorting logic for candidate instructors during slot assignment.
- **Key Operations:**
    - `rankCandidates(candidates: List<Teacher>, slot: TimeSlot): List<Teacher>`

2.3 Architectural Design Rationale
The component architecture addresses three core quality attributes:

1. **Modularity (The Litmus Test):** In accordance with the professor's architectural criteria, each component can be cleanly replaced or mock-implemented without altering surrounding modules. For example, `RankingStrategyComponent` can introduce new algorithmic strategies (e.g., Seniority-based ranking) without modifying `SchedulingComponent`.
2. **Encapsulation of Volatile Business Rules:** Singapore statutory labor rules (44h cap, 12h daily spread) are encapsulated within `AuditComplianceComponent` and `ValidationComponent`. If statutory limits are updated, only these components undergo modification.
3. **Decoupled Infrastructure Gateways:** The `NotificationComponent` and `PersistenceComponent` encapsulate external network dependencies (SMTP/SMS APIs and database drivers), preventing external library changes from polluting core business logic.

3. Detailed Design

3.1 Software Architecture Design

The system implements a classic **3-Tier Layered Architecture** augmented with **Dependency Inversion** to enforce strict separation of concerns across presentation, business logic, and persistence infrastructure.

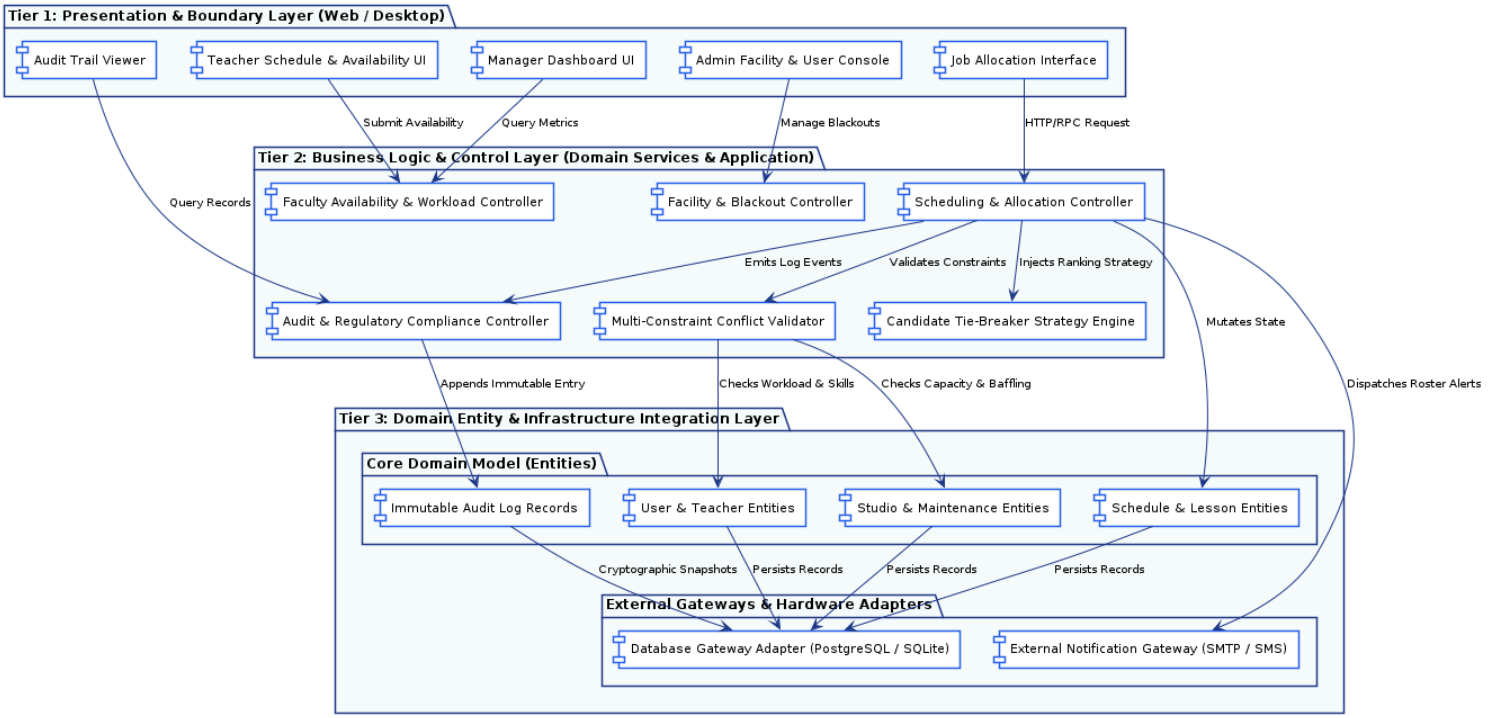


Figure 3.1: 3-Tier Software Architecture Diagram

Layer Responsibilities & Interaction Protocols:

1. **Tier 1: Presentation & Boundary Layer (Web / Desktop)**
   - **Components:** `ManagerDashboardUI`, `AllocationUI`, `TeacherUI`, `AdminUI`, `AuditLogUI`.
   - **Responsibilities:** Renders responsive UI views (375px mobile to 1920px desktop), validates form inputs client-side, and communicates with Tier 2 exclusively via service interfaces.
   - **Constraint:** Zero direct coupling to Tier 3 entities or database drivers.
2. **Tier 2: Business Logic & Control Layer (Domain Services & Application)**
   - **Components:** `AllocationController`, `ConflictValidator`, `WorkloadController`, `AvailabilityController`, `AuditLogController`, `ITieBreakerStrategy`.
   - **Responsibilities:** Orchestrates business transactions, enforces multi-constraint scheduling rules (4h block limits, instrument matching, Studio 5 drum isolation), and manages transactional workflows.
   - **Dependency Inversion:** Tier 2 defines abstract service interfaces realized by underlying controllers and infrastructure adapters.
3. **Tier 3: Domain Entity & Infrastructure Integration Layer**
   - **Components:** Domain Entities (`Teacher`, `Lesson`, `Studio`, `Schedule`, `AuditLog`), `DatabaseGateway`, and `External Notification Service`.
   - **Responsibilities:** Encapsulates enterprise business rules, maintains structural entity relationships, and manages physical data persistence and asynchronous network dispatching.

3.2 Design Pattern Evaluation & Implementation

3.2.1 Pattern Evaluation: Strategy vs. Observer

To achieve high internal software quality, two prominent Gang of Four (GoF) behavioral design patterns were evaluated against the system's operational requirements:

Option 1: GoF Strategy Pattern (Selected for Implementation)

- **Problem Context:** In `UC-02` (Allocate Lesson Slot), managers frequently require different sorting algorithms to rank candidate instructors (e.g., prioritizing staff with the largest workload deficit to ensure statutory equity, versus prioritizing staff with matching time preferences to minimize schedule friction).
- **Evaluation:** Hardcoding these sorting rules into `AllocationController` using conditional `if-else` blocks violates the **Open-Closed Principle (OCP)** and bloats controller logic. The Strategy Pattern decouples the ranking algorithms into interchangeable classes conforming to `ITieBreakerStrategy`.
- **Verdict: Selected for implementation.** Provides immediate architectural flexibility and allows managers to toggle tie-breaking heuristics dynamically at runtime.

Option 2: GoF Observer Pattern (Evaluated Alternative)

- **Problem Context:** When a weekly roster transitions from `Draft` to `Published` status (`UC-05`), multiple decoupled listeners require notification (e.g., dispatching SMS alerts to assigned teachers, updating audit logs, and refreshing the manager's published dashboard).
- **Evaluation:** While the Observer Pattern cleanly decouples subject state changes from listener actions, roster publishing is an infrequent, transactional batch operation (occurring once weekly). Introducing a full event-bus infrastructure for M2 adds unnecessary architectural overhead without substantial operational return.
- **Verdict:** Evaluated and documented as an alternative; Strategy Pattern prioritized due to higher daily operational complexity.

3.2.2 Selected Implementation: Strategy Pattern

The GoF Strategy Pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable, allowing the algorithm to vary independently from the clients that use it.

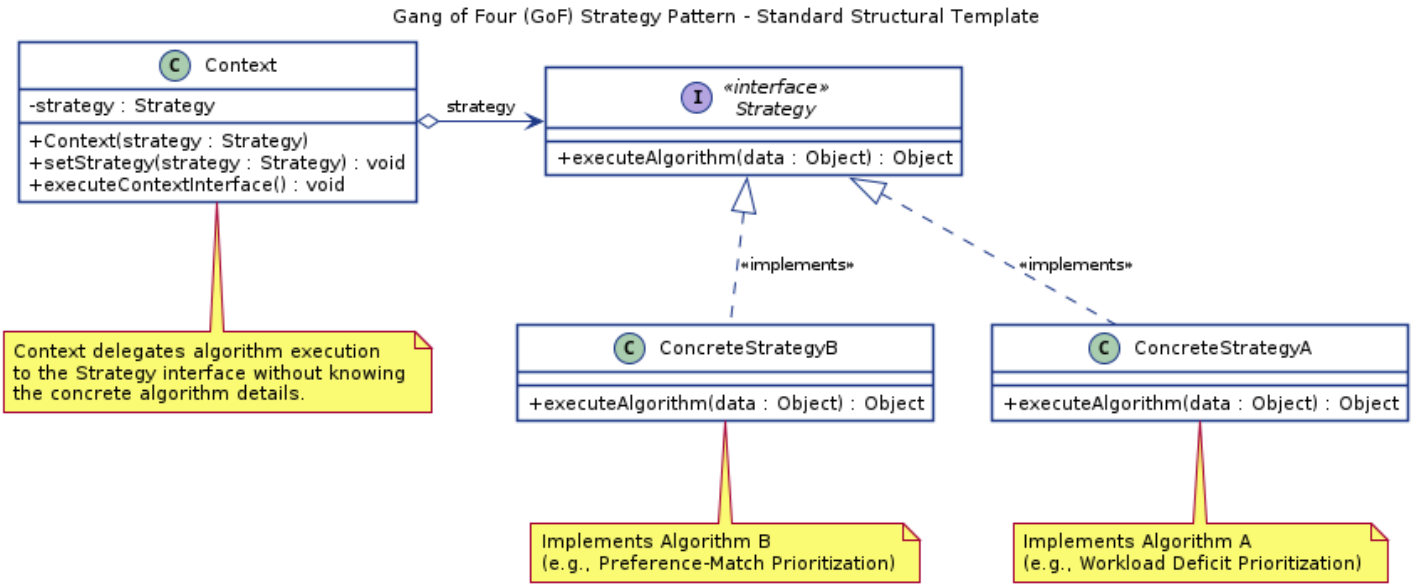


Figure 3.2: Standard GoF Strategy Pattern Structural Template

- **Context (`AllocationController`):** Maintains a reference to an `ITieBreakerStrategy` object and delegates candidate ranking to the configured strategy.
- **Strategy Interface (`ITieBreakerStrategy`):** Declares a common interface `rankCandidates(candidates: List<Teacher>, slot: TimeSlot): List<Teacher>`.
- **Concrete Strategy A (`WorkloadDeficitStrategy`):** Implements `FR-06` by sorting eligible instructors in descending order of workload deficit ($\text{Deficit} = 40.0 - \text{runningWeeklyHours}$).
- **Concrete Strategy B (`PreferenceMatchStrategy`):** Sorts eligible instructors by matching declared day preferences and consecutive 4-hour lesson clustering.

3.2.3 Structural Transformation: Before & After

To demonstrate the concrete engineering impact of applying the design pattern, Figures 3.3 and 3.4 contrast the class structure before and after refactoring:

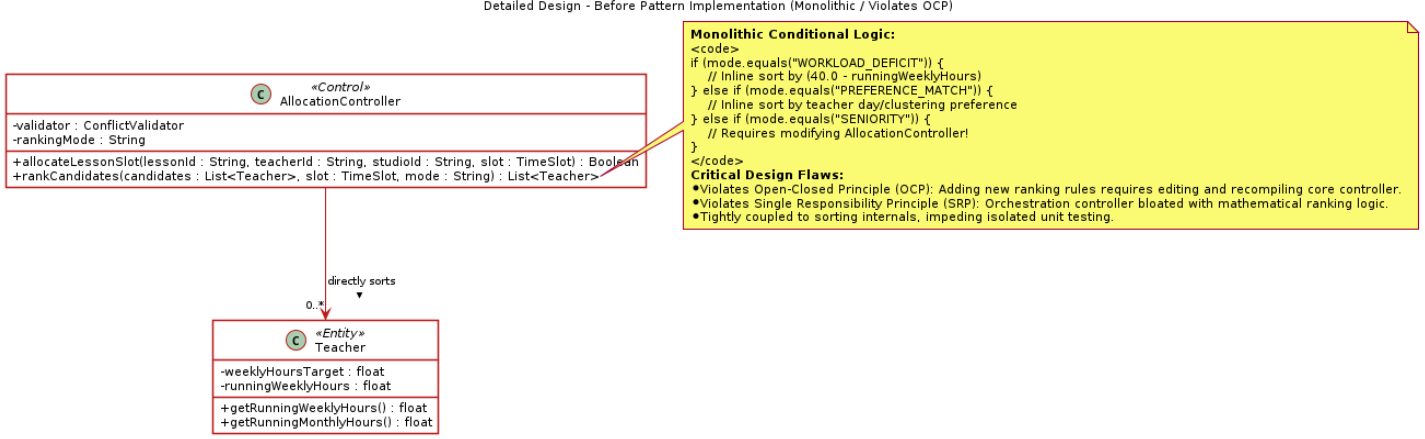


Figure 3.3: Detailed Design - Before Pattern Implementation (Monolithic / Violates OCP)

Shortcomings of "Before" Architecture: In Figure 3.3, `AllocationController` contains monolithic conditional statements (`if (mode == "WORKLOAD_DEFICIT") ... else if ...`). Adding a new policy (e.g., Seniority or Student Continuity) requires editing and recompiling core controller code, violating OCP and hindering unit testing.

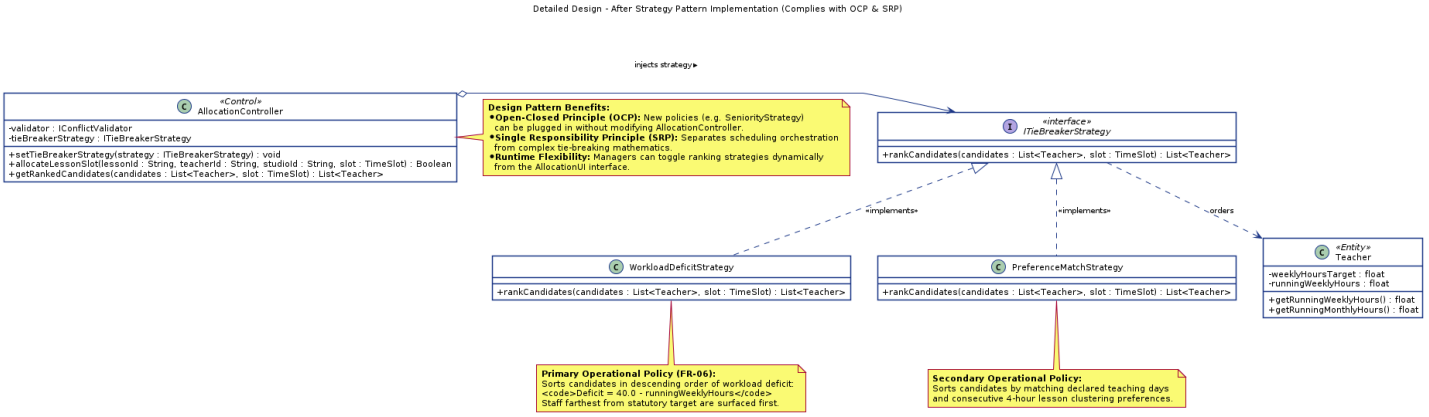


Figure 3.4: Detailed Design - After Strategy Pattern Implementation (Complies with OCP & SRP)

Benefits of "After" Architecture: In Figure 3.4, `AllocationController` depends exclusively on the `ITieBreakerStrategy` abstraction. New sorting policies are added by implementing the interface without touching a single line of controller code, achieving complete compliance with OCP and SRP.

4. Testing Plan & Verification

4.1 Black Box Testing: Decision Table Derivation

Black box testing evaluates system behavior against specification requirements without knowledge of internal code structures. The decision table technique was applied to the system's most prominent operational method:

Method Under Test: `ConflictValidator.validateAllocationRules(teacher, studio, slot)`

Step 1: Definition of Causes (Inputs / Conditions)

- **C1:** Teacher holds active certification for lesson instrument (`hasInstrumentQualification == true`).
- **C2:** Teacher marked available in submitted monthly availability grid (`isAvailableInSlot == true`).
- **C3:** Allocation preserves continuous teaching limit ($\leq 4.0\text{h}$ before 1.0h rest) (`withinContinuousLimit == true`).
- **C4:** Projected weekly hours ($\text{current} + 0.5\text{h}$) does not exceed statutory ceiling ($\leq 44.0\text{h}$) (`withinStatutoryCeiling == true`).
- **C5:** Studio instrument compatibility valid (Drums placed strictly in Studio 5) (`studioInstrumentMatch == true`).
- **C6:** Studio is unoccupied during requested timeslot (`studioUnoccupied == true`).
- **C7:** Studio is not under scheduled maintenance blackout window (`studioNotBlackedOut == true`).

Step 2: Definition of Effects (Outputs / Actions)

- **E1:** Allocation Permitted (`ValidationResult.status == APPROVED`).
- **E2:** Allocation Rejected with Specific Error Message (`ValidationResult.status == REJECTED`).
- **E3:** Advisory Overtime Soft-Warning Prompt Triggered ($> 40.0\text{h}$ and $\leq 44.0\text{h}$) (`requiresOvertimeSignOff == true`).

Step 3: Full Decision Table ($2^7 = 128$ Theoretical Combinations)

To eliminate invalid states, mandatory physical preconditions (studio occupancy and maintenance) are evaluated hierarchically. Table 4.1 details the representative truth table covering nominal paths, single-failure boundaries, and overtime advisory paths:

Table 4.1: Representative Full Decision Table Combinations

Rule ID	C1 (Qual)	C2 (Avail)	C3 (4h Block)	C4 (<=44h)	C5 (Studio Match)	C6 (Room Free)	C7 (No Blackout)	E1 (Approve)	E2 (Reject)	E3 (Overtime Alert)
R01	T	T	T	T ($\leq 40.0\text{h}$)	T	T	T	**X**		
R02	T	T	T	T ($40.5\text{h}..44.0\text{h}$)	T	T	T	**X**		**X**
R03	**F**	T	T	T	T	T	T		**X**	
R04	T	**F**	T	T	T	T	T		**X**	
R05	T	T	**F**	T	T	T	T		**X**	
R06	T	T	T	**F** ($> 44.0\text{h}$)	T	T	T		**X**	
R07	T	T	T	T	**F**	T	T		**X**	
R08	T	T	T	T	T	**F**	T		**X**	
R09	T	T	T	T	T	T	**F**		**X**	

Step 4: Boolean Algebraic Reduction

Applying prime implicant grouping where any single prerequisite failure immediately aborts validation yields the **Reduced Decision Table**:

Table 4.2: Reduced Decision Table

Rule ID	C1 (Qual)	C2 (Avail)	C3 (4h Limit)	C4 (Hours)	C5 (Studio Match)	C6 (Free)	C7 (No Blackout)	Action / Outcome
R-Nominal	T	T	T	$\leq 40.0\text{h}$	T	T	T	**E1: Approve allocation**
R-Overtime	T	T	T	$40.5\text{h}..44.0\text{h}$	T	T	T	**E1 + E3: Approve with Overtime Prompt**
R-Fail-Qual	**F**	—	—	—	—	—	—	**E2: Reject (Uncertified Instructor)**
R-Fail-Avail	T	**F**	—	—	—	—	—	**E2: Reject (Instructor Unavailable)**
R-Fail-Block	T	T	**F**	—	—	—	—	**E2: Reject (Exceeds 4h Block Limit)**
R-Fail-Cap	T	T	T	$> 44.0\text{h}$	—	—	—	**E2: Reject (Breaches 44.0h Statutory Cap)**
R-Fail-Inst	T	T	T	T	**F**	—	—	**E2: Reject (Incompatible Studio Asset)**
R-Fail-Room	T	T	T	T	T	**F**	—	**E2: Reject (Studio Collision / Occupied)**
R-Fail-Maint	T	T	T	T	T	T	**F**	**E2: Reject (Studio Under Maintenance)**

Step 5: Final Black Box Test Case Suite

Table 4.3: Final Test Cases Derived from Reduced Decision Table

Test Case ID	Linked Rule	Test Input Parameters (Teacher, Studio, Timeslot)	Expected Result	Acceptance Verification Condition
TC-BB-01	`R-Nominal`	Teacher A (Piano Cert, Avail, 28h workload), Studio 1 (Free, Open), Mon 10:00	`APPROVED`	Booking committed; teacher workload tallies increment to 28.5h.
TC-BB-02	`R-Overtime`	Teacher B (Violin Cert, Avail, 41.5h workload), Studio 2 (Free, Open), Tue 14:00	`APPROVED + WARN`	Overtime authorization prompt rendered; requires manager sign-off reason.
TC-BB-03	`R-Fail-Qual`	Teacher C (Violin Cert only), Drums Lesson, Studio 5, Wed 11:00	`REJECTED`	Error: "Instructor not certified for Drums." Allocation blocked.
TC-BB-04	`R-Fail-Avail`	Teacher A, Slot outside submitted availability grid, Studio 1, Thu 09:00	`REJECTED`	Error: "Instructor has not declared availability." Allocation blocked.
TC-BB-05	`R-Fail-Block`	Teacher A (Already scheduled 4.0h continuous block 13:00-17:00), Slot at 17:00	`REJECTED`	Error: "Continuous teaching ceiling (4.0h) reached. Mandatory 1h rest required."
TC-BB-06	`R-Fail-Cap`	Teacher B (Workload at 44.0h statutory maximum), Studio 3, Fri 16:00	`REJECTED`	Error: "Statutory 44.0-hour ceiling reached under Employment Act Part IV."

TC-BB-07	`R-Fail-Inst`	Teacher D (Drums Cert, Avail), Drums Lesson, Studio 1 (Piano only), Sat 10:00	`REJECTED`	Error: "Drums lessons must be conducted in Studio 5 (Acoustic Isolation)."
TC-BB-08	`R-Fail-Room`	Teacher A, Studio 1 (Already booked by Lesson L-102 at 10:00), Mon 10:00	`REJECTED`	Error: "Physical room collision: Studio 1 is occupied." Allocation blocked.
TC-BB-09	`R-Fail-Maint`	Teacher A, Studio 4 (Scheduled for Piano Tuning 14:00-16:00), Wed 14:30	`REJECTED`	Error: "Studio 4 is suppressed under active Maintenance Blackout Window."

4.2 White Box Testing: Basis Path & Cyclomatic Complexity

White box testing evaluates internal program logic, control structures, and execution paths. In accordance with the project specification, a prominent algorithmic method distinct from Section 4.1 was selected:

Method Under Test: `ConflictValidator.findEligibleCandidates(instrument, slot, weekSchedule)`

Step 1: Formal Line-Numbered Pseudocode (19 Lines, 5 Decision Points)

```
01: List<Teacher> findEligibleCandidates(InstrumentType instrument, TimeSlot slot, Schedule
weekSchedule) {
02:     List<Teacher> eligibleCandidates = new ArrayList<>();
03:     List<Teacher> facultyPool = getActiveFacultyPool();
04:     int i = 0;
05:     while (i < facultyPool.size()) {
06:         Teacher candidate = facultyPool.get(i);
07:         if (candidate.hasInstrumentQualification(instrument)) {
08:             if (candidate.isAvailable(slot.getDate(), slot.getStartTime())) {
09:                 if (checkContinuousTeachingLimit(candidate, slot)) {
10:                     if (candidate.getRunningWeeklyHours() + 0.5 <= 44.0) {
11:                         eligibleCandidates.add(candidate);
12:                     }
13:                 }
14:             }
15:         }
16:         i = i + 1;
17:     }
18:     return eligibleCandidates;
19: }
```

Step 2: Control Flow Graph (CFG)

Figure 4.1 models the execution control flow graph where circles represent statement blocks and directed edges denote branching transitions:

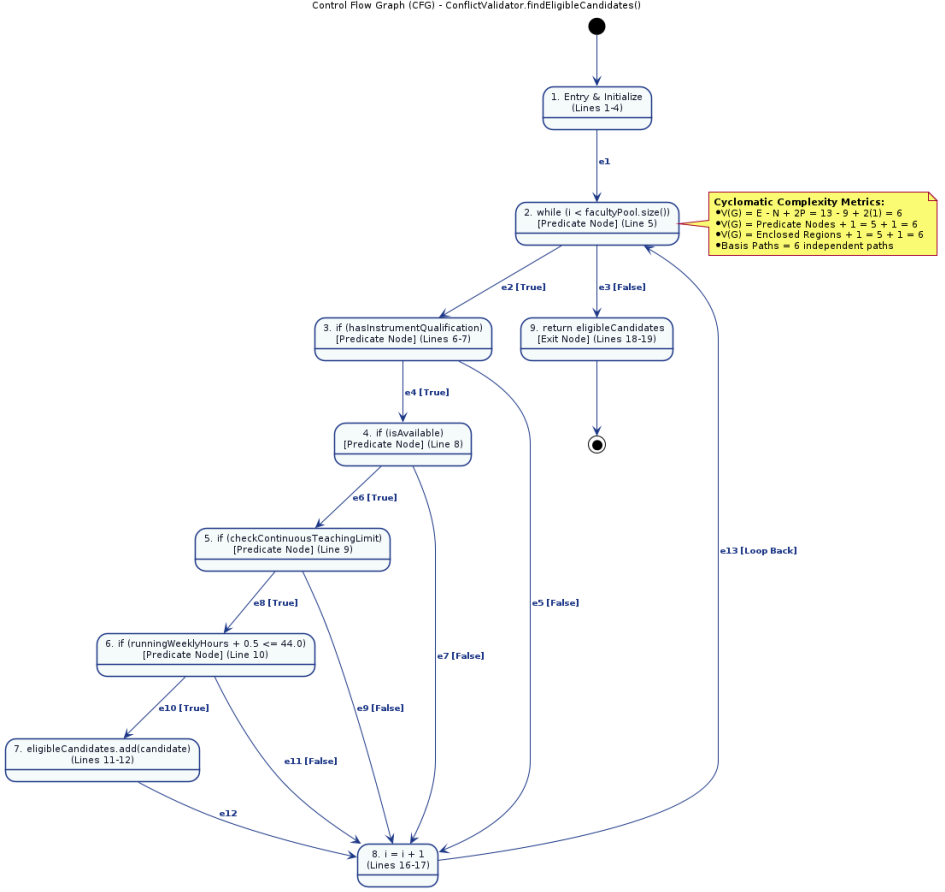


Figure 4.1: Control Flow Graph (CFG) for findEligibleCandidates()

CFG Node & Edge Mapping:

- **Node 1:** Initialization & list allocation (Lines 01–04)
- **Node 2:** Loop predicate: `while (i < facultyPool.size())` (Line 05) — *Predicate Node*
- **Node 3:** Qualification check: `if (hasInstrumentQualification)` (Lines 06–07) — *Predicate Node*
- **Node 4:** Availability check: `if (isAvailable)` (Line 08) — *Predicate Node*
- **Node 5:** Continuous limit check: `if (checkContinuousTeachingLimit)` (Line 09) — *Predicate Node*
- **Node 6:** Statutory cap check: `if (runningWeeklyHours + 0.5 <= 44.0)` (Line 10) — *Predicate Node*
- **Node 7:** Add candidate to results: `eligibleCandidates.add(candidate)` (Lines 11–12)
- **Node 8:** Loop counter increment: `i = i + 1` (Lines 16–17)
- **Node 9:** Method termination: `return eligibleCandidates` (Lines 18–19) — *Exit Node*

Graph Counts:

- Total Nodes (N) = 9
- Total Directed Edges (E) = 13
- Total Predicate Nodes (P) = 5 (Nodes 2, 3, 4, 5, 6)

Step 3: Cyclomatic Complexity Mathematical Calculation

Cyclomatic complexity $V(G)$ quantifies the number of linearly independent paths through the program. We verify $V(G)$ using all three standard software engineering methods:

Method 1: Edge-Node Formula

$$V(G) = E - N + 2P_{\text{components}} = 13 - 9 + 2(1) = 4 + 2 = \mathbf{6}$$

Method 2: Predicate Node Formula

$$V(G) = P + 1 = 5 + 1 = \mathbf{6}$$

Method 3: Enclosed Regions Formula

$$V(G) = \text{Enclosed Planar Regions} + 1 = 5 + 1 = \mathbf{6}$$

All three mathematical derivations arrive identically at $V(G) = 6$, confirming that exactly **6 linearly independent basis paths** are required to guarantee 100% statement and branch coverage.

Step 4: Independent Basis Paths

1. **Path 1 (Empty Pool):**

$$1 \longrightarrow 2 \longrightarrow 9$$

   (Loop terminates immediately when faculty pool is empty).
2. **Path 2 (Unqualified Candidate):**

$$1 \longrightarrow 2 \longrightarrow 3 \longrightarrow 8 \longrightarrow 2 \longrightarrow 9$$

   (Candidate fails instrument qualification check; bypassed to increment).
3. **Path 3 (Unavailable Candidate):**

$$1 \longrightarrow 2 \longrightarrow 3 \longrightarrow 4 \longrightarrow 8 \longrightarrow 2 \longrightarrow 9$$

   (Candidate qualified, but marked unavailable in timeslot).
4. **Path 4 (Fatigue Ceiling Breach):**

$$1 \longrightarrow 2 \longrightarrow 3 \longrightarrow 4 \longrightarrow 5 \longrightarrow 8 \longrightarrow 2 \longrightarrow 9$$

   (Candidate available, but breaches 4.0h continuous teaching ceiling).
5. **Path 5 (Statutory 44.0h Cap Breach):**

$$1 \longrightarrow 2 \longrightarrow 3 \longrightarrow 4 \longrightarrow 5 \longrightarrow 6 \longrightarrow 8 \longrightarrow 2 \longrightarrow 9$$

   (Candidate passes fatigue check, but current workload is already $\geq 43.8\text{h}$, causing projected hours to exceed 44.0h).
6. **Path 6 (Nominal Acceptance):**

$$1 \longrightarrow 2 \longrightarrow 3 \longrightarrow 4 \longrightarrow 5 \longrightarrow 6 \longrightarrow 7 \longrightarrow 8 \longrightarrow 2 \longrightarrow 9$$

   (Candidate passes all filters and is added to `eligibleCandidates`).

Step 5: White Box Test Cases

Table 4.4: Basis Path Test Cases for findEligibleCandidates()

Test ID	Path Traversed	Input Faculty Data & Condition Setup	Expected Path Execution & Result
TC-WB-01	Path 1 ($1 \to 2 \to 9$)	`facultyPool = []` (Empty list)	Node 2 evaluates false. Returns empty list `[]`.
TC-WB-02	Path 2 ($1 \to 2 \to 3 \to 8 \to 2 \to 9$)	Candidate has `qualifications = [PIANO]`, searching for `DRUMS`	Node 3 evaluates false, jumps directly to Node 8. Candidate omitted.
TC-WB-03	Path 3 ($1 \to 2 \to 3 \to 4 \to 8 \to 2 \to 9$)	Candidate certified for Piano, but slot is outside submitted grid	Node 4 evaluates false, jumps to Node 8. Candidate omitted.

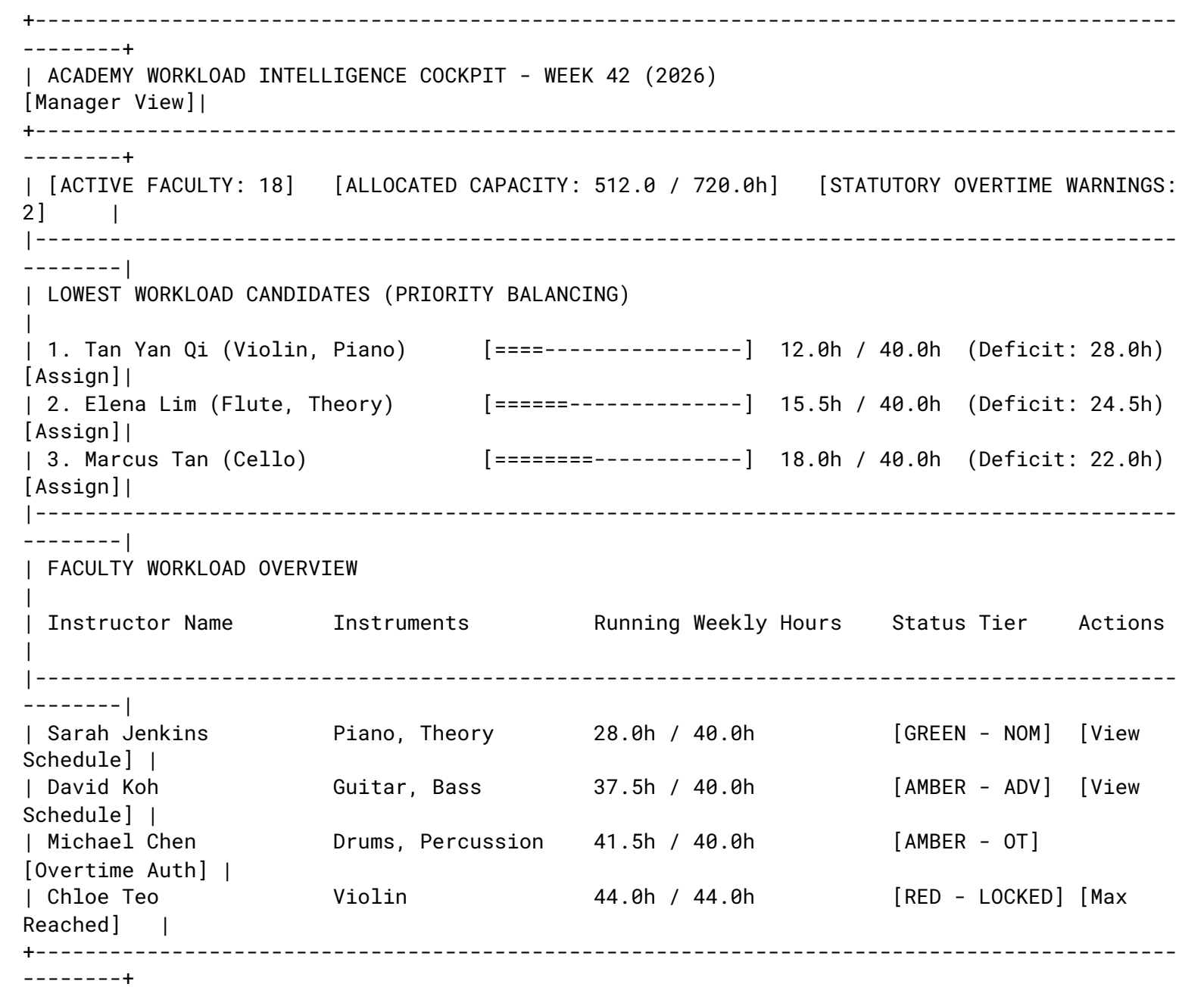
TC-WB-04	Path 4 ($1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 8 \rightarrow 2 \rightarrow 9$)	Candidate certified & available, but has taught 4.0h continuous	Node 5 evaluates false (fatigue limit breached), jumps to Node 8.
TC-WB-05	Path 5 ($1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 8 \rightarrow 2 \rightarrow 9$)	Candidate certified, available, fatigue OK, but `runningWeeklyHours = 44.0h`	Node 6 evaluates false ($44.0 + 0.5 > 44.0$), jumps to Node 8.
TC-WB-06	Path 6 ($1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 2 \rightarrow 9$)	Candidate certified, available, fatigue OK, and `runningWeeklyHours = 32.0h`	Traverses through Node 7. Candidate appended to `eligibleCandidates`.

5. Wireframe & User Interaction Design

The user interface was engineered to bridge functional requirements with human-centered ergonomics, ensuring frictionless decision-support across all 17 finalized use cases. Wireframes were designed using high-fidelity Figma components and exported via automated browser tooling.

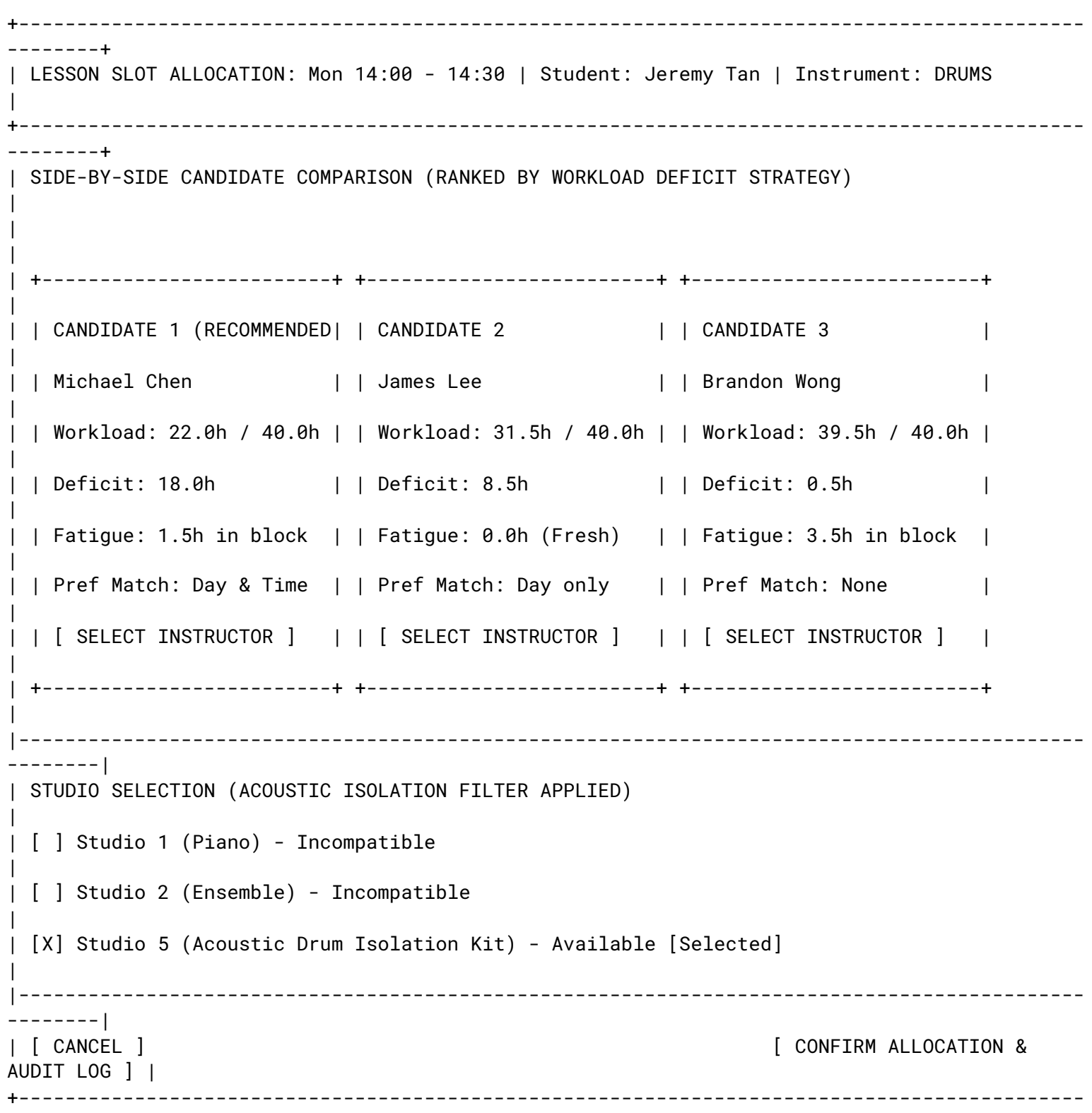
5.1 Manager Scheduling & Outlier Intelligence View

- **Linked Use Case:** UC-01 (View Workload Dashboard & Outliers)
- **Supported Requirements:** FR-01, FR-02, FR-03, NFR-USE-02.
- **Screen Purpose:** Serves as the operational landing cockpit for school managers, summarizing real-time faculty capacity across active instructors and proactively highlighting staffing inequities prior to weekly roster creation.
- **Key User Interactions:**
    1. **Workload Progress Indicators:** Renders real-time capacity meters color-coded according to NFR-USE-02 (Green $< 35.0\text{h}$, Amber $35.0\text{h}..40.0\text{h}$, Red $> 40.0\text{h}$).
    2. **Top 3 Under-Allocated Outliers:** Automatically bubbles up the 3 instructors with the lowest allocated hours, enabling managers to assign pending lessons with one click.
    3. **Statutory Ceiling Warning Badges:** Highlights any instructor approaching or reaching the statutory 44.0-hour maximum with a red lock badge, blocking additional assignments.



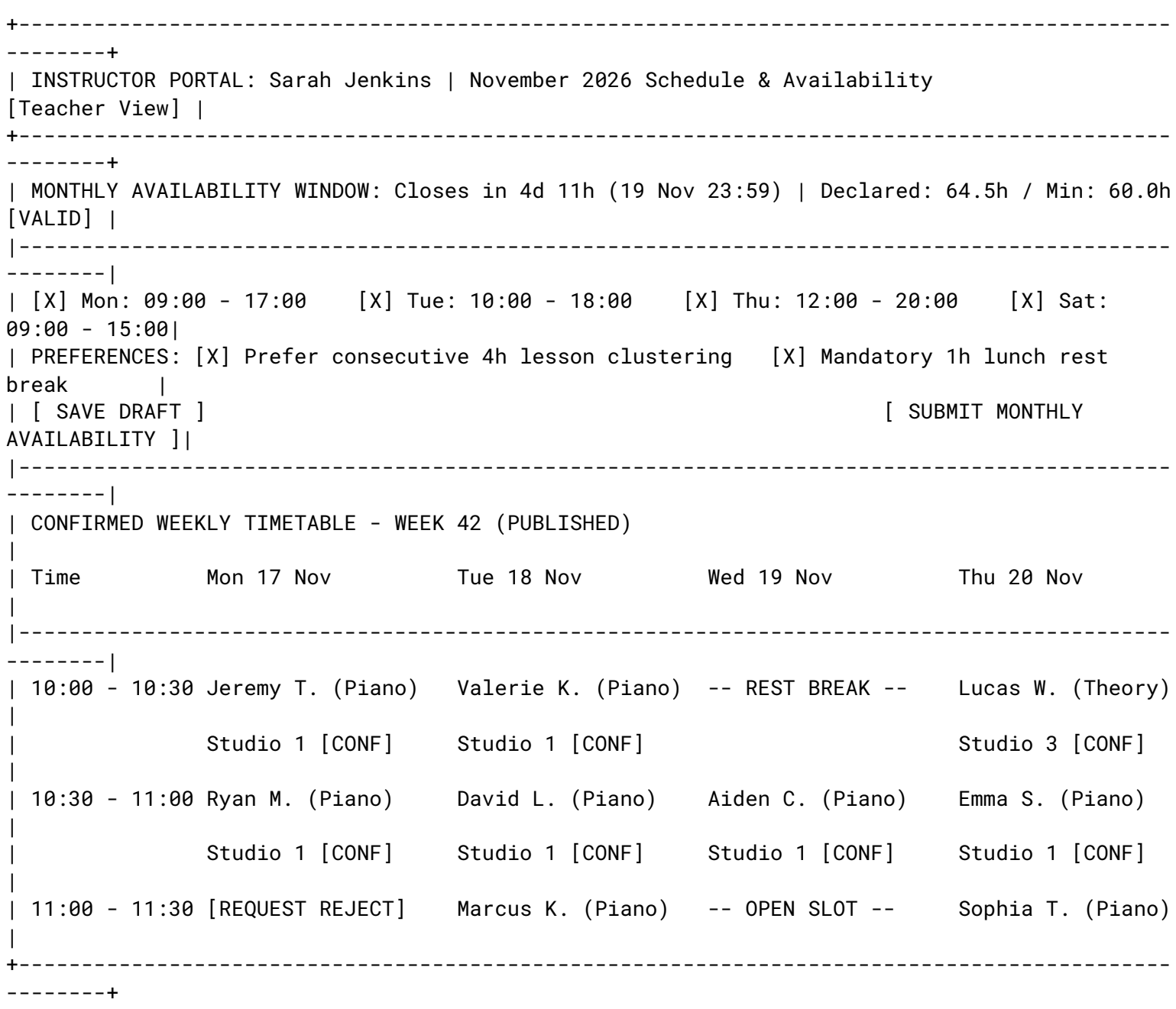
5.2 Interactive Allocation Matrix & Conflict Feedback View

- **Linked Use Case:** `UC-02` (Allocate Lesson Slot), `UC-03` (Review & Re-allocate), `UC-06` (Emergency Substitute).
- **Supported Requirements:** `FR-04`, `FR-05`, `FR-06`, `FR-17`, `FR-20`.
- **Screen Purpose:** Provides managers with side-by-side decision support when placing a 30-minute student lesson slot, executing sub-300ms conflict checks against qualifications, availability, 4-hour fatigue limits, and room compatibility.
- **Key User Interactions:**
    1. **Candidate Comparison Panel:** Presents up to 3 pre-filtered candidates side-by-side with running weekly hours, monthly totals, preferences, and studio location for that date.
    2. **Automated Tie-Breaking Strategy:** Injects `ITieBreakerStrategy` (Workload Deficit or Preference Match), surfacing the most eligible candidate with a "Recommended" badge.
    3. **Studio Isolation Enforcement:** Restricts studio pickers dynamically (Studio 5 locked for Drums; maintenance windows disabled).
    4. **Advisory Overtime Modal:** If assigning the lesson pushes the instructor between 40.5h and 44.0h, triggers an advisory confirmation modal requiring mandatory justification before commit.



5.3 Teacher Availability & Timetable Inspection View

- **Linked Use Case:** UC-07 (Submit Availability), UC-08 (Submit Preferences), UC-09 (View Schedule), UC-10 (Rejection).
- **Supported Requirements:** FR-07, FR-08, FR-09, FR-10, FR-11, NFR-SEC-01.
- **Screen Purpose:** Provides instructors with an ergonomic, responsive portal to declare recurring monthly availability, submit consecutive clustering preferences, inspect published timetables, and initiate rejection tickets if necessary.
- **Key User Interactions:**
    1. **Availability Grid (30-min increments):** Visual click-and-drag grid; enforces the minimum 60.0-hour declaration rule before allowing submission.
    2. **19th 23:59 Cutoff Clock:** Displays a real-time countdown timer. Edits prior to cutoff save automatically; edits after cutoff require manager review (UC-04).
    3. **Masked Student Contact PII:** Displays student first names and instrument levels, strictly masking contact details in compliance with Singapore PDPA Section 24.
    4. **Job Rejection Modal:** Displays a consultation advisory modal, captures mandatory reason codes, and forwards rejected slots to the manager exception queue (UC-03).



5.4 Audit Log Console & Governance View

- **Linked Use Case:** UC-11 (Inspect System Audit Log), UC-17 (Review Labor Compliance).
- **Supported Requirements:** FR-20, FR-21, NFR-COM-01, NFR-COM-02.

- **Screen Purpose:** Furnishes school administrators and compliance officers with an unalterable chronological ledger of all scheduling mutations, overtime overrides, and system access changes.
- **Key User Interactions:**
    1. **Multi-Parameter Filtering:** Enables instant querying by date range, staff ID, action type (`ALLOCATION_COMMIT`, `OVERTIME_OVERRIDE`, `REJECTION_RESOLVED`), or entity ID.
    2. **Cryptographic Integrity Badges:** Validates and displays SHA-256 tamper-evident integrity hashes for each record.
    3. **Export Compliance Bundle:** Generates exportable CSV/PDF audit reports satisfying Singapore Ministry of Manpower 24-month record retention mandates.

```
+-------------------------------------------------------------------------------------------------------+
| REGULATORY COMPLIANCE & AUDIT TRAIL CONSOLE - SINGAPORE MOM GOVERNANCE          [Admin / Director]|
+-------------------------------------------------------------------------------------------------------+
| FILTERS: [Date: Last 30 Days]  [Actor: All Staff]  [Action: OVERTIME_OVERRIDE]  [Apply Filter]     |
|-------------------------------------------------------------------------------------------------------|
| Timestamp           Actor ID     Action Type          Target Entity  Details / Justification       |
|-------------------------------------------------------------------------------------------------------|
| 2026-11-15 14:22:04 MGR-01 (MT)  OVERTIME_OVERRIDE    TCH-04 (MC)    Assigned L-409; Hours: 41.5h  |
|                                                                       Reason: Acute sub for flu     |
| 2026-11-14 09:12:33 MGR-02 (WK)  ALLOCATION_COMMIT    L-388          Assigned TCH-01, Studio 1     |
| 2026-11-12 18:45:11 IT-01 (AW)   MAINT_BLACKOUT_ADD   STU-04         Tuning blackout 14:00-16:00   |
| 2026-11-10 11:05:49 TCH-02 (EL)  REJECTION_SUBMIT     L-352          Reason: Medical appointment   |
| 2026-11-10 11:30:22 MGR-01 (MT)  REALLOCATION_RESOLVE L-352          Reassigned to TCH-05 (Deficit)|
|-------------------------------------------------------------------------------------------------------|
| [ EXPORT CSV AUDIT LEDGER ]                              [ EXPORT MOM COMPLIANCE CERTIFICATE ]    |
+-------------------------------------------------------------------------------------------------------+
```

6. References

1. **Booch, G., Rumbaugh, J., & Jacobson, I.** (2005). *The Unified Modeling Language User Guide* (2nd ed.). Addison-Wesley Professional.
2. **Gamma, E., Helm, R., Johnson, R., & Vlissides, J.** (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
3. **McCabe, T. J.** (1976). A Complexity Measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308–320.
4. **Ministry of Manpower Singapore.** (2026). *Employment Act (Chapter 91) — Part IV: Hours of Work, Rest Days and Other Conditions of Service*. Singapore Statutes Online.
5. **Personal Data Protection Commission Singapore.** (2024). *Advisory Guidelines on Key Concepts in the Personal Data Protection Act (PDPA Section 24: Protection Obligation)*.
6. **Pressman, R. S., & Maxim, B. R.** (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.
7. **World Wide Web Consortium (W3C).** (2023). *Web Content Accessibility Guidelines (WCAG) 2.2 — Rule 1.4.1: Use of Color*. W3C Recommendation.